

Bases de datos

UD6. Consultas de recuperación multitabla en SQL

6.3. JOIN



JOIN

Un JOIN es una composición entre dos tablas. Es decir, devuelve una nueva tabla compuesta por información de las tablas indicadas.

Es similar al producto cartesiano, con la diferencia de que la condición para componer la nueva tabla no aparece en el WHERE, sino en el propio JOIN.

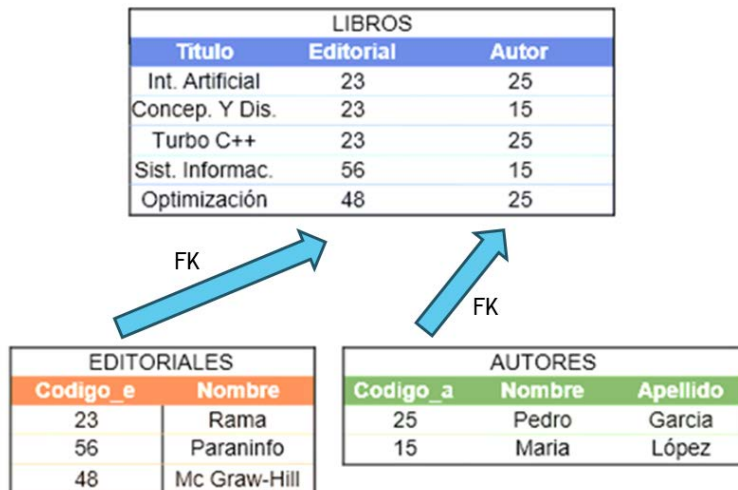
Existen varios tipos de JOIN:

- ▶ Composición interna: INNER JOIN (y NATURAL JOIN que no se explicará y se desaconseja su uso).
- ▶ Composición externa:
 - ▶ De la tabla izquierda: LEFT OUTER JOIN
 - ▶ De la tabla derecha: RIGHT OUTER JOIN
 - ▶ Completa: FULL OUTER JOIN
- ▶ Producto cartesiano: CROSS JOIN.

CROSS JOIN

Es similar al producto cartesiano que ya conocemos. En vez de separar las tablas por comas, se separan por las palabras CROSS JOIN.

```
SELECT *  
FROM LIBROS  
CROSS JOIN EDITORIALES  
CROSS JOIN AUTORES;
```



Título	Editorial	Autor	Codigo_e	Nombre	Codigo_a	Nombre	Apellido
Int. Artificial	23	25	23	Rama	25	Pedro	Garcia
Int. Artificial	23	25	56	Paraninfo	25	Pedro	Garcia
Int. Artificial	23	25	48	Mc Graw-Hill	25	Pedro	Garcia
Concep. Y Dis.	23	15	23	Rama	25	Pedro	Garcia
Concep. Y Dis.	23	15	56	Paraninfo	25	Pedro	Garcia
Concep. Y Dis.	23	15	48	Mc Graw-Hill	25	Pedro	Garcia
Turbo C++	23	25	23	Rama	25	Pedro	Garcia
Turbo C++	23	25	56	Paraninfo	25	Pedro	Garcia
Turbo C++	23	25	48	Mc Graw-Hill	25	Pedro	Garcia
Sist. Informac.	56	15	23	Rama	25	Pedro	Garcia
Sist. Informac.	56	15	56	Paraninfo	25	Pedro	Garcia
Sist. Informac.	56	15	48	Mc Graw-Hill	25	Pedro	Garcia
Optimización	48	25	23	Rama	25	Pedro	Garcia
Optimización	48	25	56	Paraninfo	25	Pedro	Garcia
Optimización	48	25	48	Mc Graw-Hill	25	Pedro	Garcia
Int. Artificial	23	25	23	Rama	15	Maria	López
Int. Artificial	23	25	56	Paraninfo	15	Maria	López
Int. Artificial	23	25	48	Mc Graw-Hill	15	Maria	López
Concep. Y Dis.	23	15	23	Rama	15	Maria	López
Concep. Y Dis.	23	15	56	Paraninfo	15	Maria	López
Concep. Y Dis.	23	15	48	Mc Graw-Hill	15	Maria	López
Turbo C++	23	25	23	Rama	15	Maria	López
Turbo C++	23	25	56	Paraninfo	15	Maria	López
Turbo C++	23	25	48	Mc Graw-Hill	15	Maria	López
Sist. Informac.	56	15	23	Rama	15	Maria	López
Sist. Informac.	56	15	56	Paraninfo	15	Maria	López
Sist. Informac.	56	15	48	Mc Graw-Hill	15	Maria	López
Optimización	48	25	23	Rama	15	Maria	López
Optimización	48	25	56	Paraninfo	15	Maria	López
Optimización	48	25	48	Mc Graw-Hill	15	Maria	López

INNER JOIN

El INNER JOIN es una forma de realizar una composición entre 2 tablas para obtener otra.

Al igual que en el producto cartesiano, combina todas las filas de la tabla indicada en primer lugar, con todas las filas de la tabla indicada en segundo lugar. Luego, para cada una de las filas resultados, aplica la condición (relación) indicada en el INNER JOIN después de la palabra ON.

```
tabla1 INNER JOIN tabla2 ON condición;
```

La condición puede ser de cualquier tipo (siempre que al aplicarse sobre las filas combinadas devuelva un valor booleano (cierto o falso). Por eso se pueden utilizar las columnas tanto de tabla, como de tabla2.

Normalmente se utiliza para comparar una columna de una tabla, con una columna de otra tabla2, que suelen ser la clave primaria (T1.campo1), y la clave foránea (T2.campo4).

Por ejemplo:

```
SELECT T1.*  
FROM tabla1 T1 INNER JOIN tabla2 T2 ON T1.campo1 = T2.campo4;
```

INNER JOIN

LIBROS			
Codigo	Titulo	Num_paginas	Editorial
34587	Int. Artificial	50	Paraninfo
1022305	Concep. Y Dis.	48	Rama
493942	Turbo C++	125	Mc Graw-Hill
45307	Virus Informát.	50	NULL
112313	Sist. Informac.	358	Rama

EDITORIALES			
Nombre_e	Direccion	Pais	Ciudad
Universal Books	Brown Sq. 23	EEUU	Los Ángeles
Rama	Canillas, 144	España	Madrid
Mc Graw-Hill	Basauri, 17	España	Madrid
Paraninfo	Virtudes, 7	España	Madrid



Vamos a usar el modelo para ver el siguiente ejemplo:

```
SELECT *  
FROM LIBROS L INNER JOIN EDITORIALES E ON L.editorial = E.nombre_e;
```

El resultado es

Codigo	Titulo	Num_paginas	Editorial	Nombre_e	Direccion	Pais	Ciudad
34590	Int. Artificial	50	Paraninfo	Paraninfo	Virtudes, 7	España	Madrid
1022306	Concep. Y Dis.	48	Rama	Rama	Canillas, 144	España	Madrid
493944	Turbo C++	125	Mc Graw-Hill	Mc Graw-Hill	Basauri, 17	España	Madrid
112314	Sist. Informac.	358	Rama	Rama	Canillas, 144	España	Madrid

En este caso, se asocia cada libro con la editorial a la que pertenece y se muestran todos los campos (*). Es decir, para cada libro se muestran los datos del libro y de la editorial. El libro de editorial nula no aparece en los resultados puesto que no se puede relacionar con ningún valor de la tabla EDITORIALES.

La siguiente consulta es equivalente y daría el mismo resultado;

```
SELECT *  
FROM LIBROS L, EDITORIALES E  
WHERE L.editorial = E.nombre_e;
```

INNER JOIN

LIBROS			
Codigo	Titulo	Num_paginas	Editorial
34587	Int. Artificial	50	Paraninfo
1022305	Concep. Y Dis.	48	Rama
493942	Turbo C++	125	Mc Graw-Hill
45307	Virus Informát.	50	NULL
112313	Sist. Informac.	358	Rama

EDITORIALES			
Nombre_e	Direccion	Pais	Ciudad
Universal Books	Brown Sq. 23	EEUU	Los Ángeles
Rama	Canillas, 144	España	Madrid
Mc Graw-Hill	Basauri, 17	España	Madrid
Paraninfo	Virtudes, 7	España	Madrid

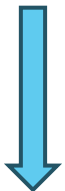
Se puede escribir la consulta de distintas formas y obtener el mismo resultado. La diferencia radica en cómo ejecuta el planificador las diferentes consultas. Esto puede llevar a diferencias de eficiencia o tiempo de ejecución, así como el orden de las columnas en el resultado.

```
SELECT *  
FROM LIBROS L INNER JOIN EDITORIALES E ON L.editorial = E.nombre_e;
```

Codigo	Titulo	Num_paginas	Editorial
34590	Int. Artificial	50	Paraninfo
1022306	Concep. Y Dis.	48	Rama
493944	Turbo C++	125	Mc Graw-Hill
112314	Sist. Informac.	358	Rama

Nombre_e	Direccion	Pais	Ciudad
Paraninfo	Virtudes, 7	España	Madrid
Rama	Canillas, 144	España	Madrid
Mc Graw-Hill	Basauri, 17	España	Madrid
Rama	Canillas, 144	España	Madrid

```
SELECT *  
FROM EDITORIALES E INNER JOIN LIBROS L ON L.editorial = E.nombre_e;
```



Nombre_e	Direccion	Pais	Ciudad
Paraninfo	Virtudes, 7	España	Madrid
Rama	Canillas, 144	España	Madrid
Mc Graw-Hill	Basauri, 17	España	Madrid
Rama	Canillas, 144	España	Madrid

Codigo	Titulo	Num_paginas	Editorial
34590	Int. Artificial	50	Paraninfo
1022306	Concep. Y Dis.	48	Rama
493944	Turbo C++	125	Mc Graw-Hill
112314	Sist. Informac.	358	Rama

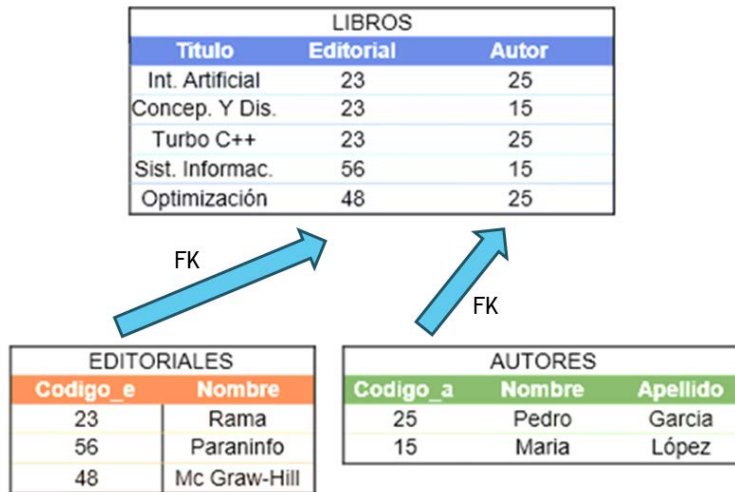
```
SELECT *  
FROM EDITORIALES E INNER JOIN LIBROS L ON E.nombre_e = L.editorial;
```



Codigo	Titulo	Num_paginas	Editorial
34590	Int. Artificial	50	Paraninfo
1022306	Concep. Y Dis.	48	Rama
493944	Turbo C++	125	Mc Graw-Hill
112314	Sist. Informac.	358	Rama

INNER JOIN

En nuestro ejemplo de las tres tablas:



```
SELECT *  
FROM LIBROS L  
      INNER JOIN EDITORIALES E ON L.editorial = E.codigo_e  
      INNER JOIN AUTORES A ON L.autor = A.codigo_a;
```

Se presentan 2 INNER JOIN. En primer lugar, combina cada libro con la editorial del libro. En segundo lugar, combina el resultado previo con el autor del libro.

Título	Editorial	Autor	Codigo_e	Nombre	Codigo_a	Nombre	Apellido
Int. Artificial	23	25	23	Rama	25	Pedro	Garcia
Turbo C++	23	25	23	Rama	25	Pedro	Garcia
Optimización	48	25	48	Mc Graw-Hill	25	Pedro	Garcia
Concep. Y Dis.	23	15	23	Rama	15	Maria	López
Sist. Informac.	56	15	56	Paraninfo	15	Maria	López

INNER JOIN

Aunque su uso más habitual es para unir una tabla con clave foránea con la tabla que tiene la clave primaria, se pueden utilizar de otras formas. Por ejemplo, en el siguiente modelo con dos tablas que no están relacionadas por FK queremos saber, por cada comprador todos los vendedores que han vendido menos cantidad de lo que ellos han comprado:

Compradores			Vendedores	
Codigo	Nombre	Num_compras	Nombre	Num_ventas
34587	Pedro	50	Luis	65
1022305	Maria	48	Juana	15
493942	Juana	125	Maria	23
45307	Lola	50	David	45
21566	Carlos	30	Amparo	48
12548	Miguel	50	Carlos	56
568978	Antonio	70	Teresa	48

```
SELECT C.nombre AS nombreComprador,
       C.num_compras,
       V.nombre AS nombreVendedor,
       V.num_ventas
FROM   compradores C
       INNER JOIN vendedores V
         ON C.num_compras >
            V.num_ventas;
```

nombreComprador	Num_compras	nombreVendedor	Num_ventas
Pedro	50	Juana	15
Pedro	50	Maria	23
Pedro	50	David	45
Pedro	50	Amparo	48
Pedro	50	Teresa	48
Maria	48	Juana	15
Maria	48	Maria	23
Maria	48	David	45
Juana	125	Luis	65
Juana	125	Juana	15
Juana	125	Maria	23
Juana	125	David	45
Juana	125	Amparo	48
Juana	125	Carlos	56
Juana	125	Teresa	48
Lola	50	Juana	15
Lola	50	Maria	23
Lola	50	David	45
Lola	50	Amparo	48
Lola	50	Teresa	48
Carlos	30	Juana	15
Carlos	30	Maria	23
Miguel	50	Juana	15
Miguel	50	Maria	23
Miguel	50	David	45
Miguel	50	Amparo	48
Miguel	50	Teresa	48
Antonio	70	Luis	65
Antonio	70	Juana	15
Antonio	70	Maria	23
Antonio	70	David	45
Antonio	70	Amparo	48
Antonio	70	Carlos	56

INNER JOIN: RELACIONES REFLEXIVAS

En cualquier consulta con más de una tabla se puede utilizar dos veces la misma tabla. En este caso, siempre es necesario el uso de alias para las diferentes ocurrencias de la tabla.

Por ejemplo, imaginemos que a partir de la tabla siguiente, necesitamos saber los empleados que tienen jefe junto con el código y nombre de sus respectivos jefes:

EMPLEADOS		
id	nombre	jefe
1	Maria	NULL
2	Lucas	1
3	David	1
4	Laura	NULL
5	Cristina	4



```
SELECT E.id AS idEmpleado,  
       E.nombre AS nombreEmpleado,  
       J.id AS idJefe,  
       J.nombre AS nombreJefe  
FROM EMPLEADOS E  
     INNER JOIN EMPLEADOS J  
     ON E.jefe = J.id;
```



idEmpleado	nombreEmpleado	idJefe	nombreJefe
2	Lucas	1	Maria
3	David	1	Maria
5	Cristina	4	Laura

Nótese que ni María ni Laura aparecen en el resultado como empleados, porque no hay ningún valor en la FK del empleado jefe.

INNER JOIN: VALORES NULL

En la comparación que aparece en un JOIN (de cualquier tipo) dos valores nulos se consideran distintos. Veamos el ejemplo de una consulta en la que queremos ver los empleados que tienen el mismo jefe

EMPLEADOS		
id	nombre	jefe
1	Maria	NULL
2	Lucas	1
3	David	1
4	Laura	NULL
5	Cristina	4



```
SELECT E1.id AS idEmpleado1,  
       E1.nombre AS nombreEmpleado1,  
       E2.id AS idEmpleado2,  
       E2.nombre AS nombreEmpleado2  
FROM EMPLEADOS E1  
     INNER JOIN EMPLEADOS E2  
     ON E1.jefe = E2.jefe ;
```



idEmpleado1	nombreEmpleado1	idEmpleado2	nombreEmpleado2
2	Lucas	2	Lucas
2	Lucas	3	David
3	David	2	Lucas
3	David	3	David
5	Cristina	5	Cristina

Se compara cada fila de empleado con todas las filas, y por tanto, también se compara cada fila consigo misma.

Sin embargo, no aparece ninguna fila con jefe nulo, ya que NULL <> NULL.

INNER JOIN: VALORES NULL

Si en el ejemplo anterior se desea quitar los empleados que aparecen relacionados consigo mismo, tenemos dos opciones:

1. En el JOIN:

```
SELECT E1.id AS idEmpleado1,  
       E1.nombre AS nombreEmpleado1,  
       E2.id AS idEmpleado2,  
       E2.nombre AS nombreEmpleado2  
FROM EMPLEADOS E1  
     INNER JOIN EMPLEADOS E2  
     ON (E1.jefe = E2.jefe  
         AND E1.id<> E2.id);
```



idEmpleado1	nombreEmpleado1	idEmpleado2	nombreEmpleado2
2	Lucas	3	David
3	David	2	Lucas

2. En el WHERE:

```
SELECT E1.id AS idEmpleado1,  
       E1.nombre AS nombreEmpleado1,  
       E2.id AS idEmpleado2,  
       E2.nombre AS nombreEmpleado2  
FROM EMPLEADOS E1  
     INNER JOIN EMPLEADOS E2  
     ON E1.jefe = E2.jefe  
WHERE E1.id<> E2.id;
```



idEmpleado1	nombreEmpleado1	idEmpleado2	nombreEmpleado2
2	Lucas	3	David
3	David	2	Lucas

En este ejemplo, el resultado es el mismo, pero no siempre es así. Hay que tener en cuenta el orden de ejecución de las partes del SELECT, ya que, después del JOIN, y antes del WHERE se ejecutan el resto de operaciones indicadas en la parte FROM.

OUTER JOIN

En este tipo de composición, se garantiza que todas las filas de la tabla indicada aparecerán al menos una vez en el resultado, independientemente de la condición del JOIN. La palabra OUTER es opcional, y no suele ponerse.

Existen 3 casos, que se corresponden con cada una de las posibilidades de tabla indicada:

- ▶ LEFT OUTER JOIN: Se garantiza que aparecerán todas las filas de la tabla de la izquierda (la que aparece antes de las palabras LEFT JOIN).
- ▶ RIGHT OUTER JOIN: Se garantiza que aparecerán todas las filas de la tabla de la derecha (la que aparece después de las palabras RIGHT JOIN).
- ▶ FULL OUTER JOIN: Se garantiza que aparecerán todas las filas de la tabla de la izquierda y de la tabla de la derecha.

OUTER JOIN

El funcionamiento de un OUTER JOIN es muy similar al del INNER JOIN:

1. Al igual que en el producto cartesiano, combina todas las filas de la tabla indicada en primer lugar (lado izquierdo o antes del OUTER JOIN), con todas las filas de la tabla indicada en segundo lugar (lado derecho o después del OUTER JOIN).
2. Luego, para cada una de las filas resultados, aplica la condición indicada en el OUTER JOIN, después de la palabra ON.
3. Por último y de forma diferente al INNER JOIN, comprueba que cada fila del lado izquierdo (en un LEFT JOIN), del lado derecho (RIGHT JOIN) o de ambos lados (FULL JOIN) aparece en el conjunto de resultados. Por cada fila que no aparece, añade una fila más a los resultados, que es la unión de esa fila, y valores NULL para el resto de columnas (correspondiente a la tabla con la que no ha encontrado relación).

LEFT JOIN

En este caso se mostrarán todos los registros de la tabla de la izquierda y aquellos de la derecha para los que haya algún valor asociado. Si no hay relación, se añade la parte derecha con todos sus campos a NULL.

Ejemplo: En el modelo Comprador/Vendedor, se desea mostrar todos los comprobadores, con su número de compras, y si además han hecho ventas, el número de ventas.

Compradores		
Codigo	Nombre	Num_compras
34587	Pedro	50
1022305	Maria	48
493942	Juana	125
45307	Lola	50
21566	Carlos	30
12548	Miguel	50
568978	Antonio	70

Vendedores		
Codigo	Nombre	Num_ventas
65154	Luis	65
132155	Juana	15
548965	Maria	23
213564	David	45
568655	Amparo	48
215685	Carlos	56
995485	Teresa	48



```
SELECT *  
FROM compradores C  
LEFT JOIN vendedores V  
ON C.nombre = V.nombre;
```



c.Codigo	c.Nombre	Num_compras	v.Codigo	v.Nombre	Num_ventas
34587	Pedro	50	NULL	NULL	NULL
1022305	Maria	48	548965	Maria	23
493942	Juana	125	132155	Juana	15
45307	Lola	50	NULL	NULL	NULL
21566	Carlos	30	215685	Carlos	56
12548	Miguel	50	NULL	NULL	NULL
568978	Antonio	70	NULL	NULL	NULL

RIGHT JOIN

En este caso se mostrarán todos los registros de la tabla de la derecha y aquellos de la izquierda para los que haya algún valor asociado. Si no hay relación, se añade la parte izquierda con todos sus campos a NULL.

Ejemplo: En el modelo Comprador/Vendedor, se desea mostrar todos los vendedores, con su número de ventas, y si además han hecho compras, el número de compras.

Compradores		
Codigo	Nombre	Num_compras
34587	Pedro	50
1022305	Maria	48
493942	Juana	125
45307	Lola	50
21566	Carlos	30
12548	Miguel	50
568978	Antonio	70

Vendedores		
Codigo	Nombre	Num_ventas
65154	Luis	65
132155	Juana	15
548965	Maria	23
213564	David	45
568655	Amparo	48
215685	Carlos	56
995485	Teresa	48



```
SELECT *  
FROM compradores C  
RIGHT JOIN vendedores V  
ON C.nombre = V.nombre;
```



c.Codigo	c.Nombre	Num_compras	v.Codigo	v.Nombre	Num_ventas
NULL	NULL	NULL	65154	Luis	65
493942	Juana	125	132155	Juana	15
1022305	Maria	48	548965	Maria	23
NULL	NULL	NULL	213564	David	45
NULL	NULL	NULL	568655	Amparo	48
21566	Carlos	30	215685	Carlos	56
NULL	NULL	NULL	995485	Teresa	48

FULL JOIN

En este caso se mostrarán todos los registros de la tabla de la derecha y aquellos de la izquierda para los que haya algún valor asociado, rellenando la parte de la derecha con los NULL correspondientes. A continuación, hará lo mismo con la tabla de la derecha.

Ejemplo: En el modelo Comprador/Vendedor, se desea mostrar todos los comprobadores y vendedores, con su número de compras y su número de ventas.

Compradores		
Codigo	Nombre	Num_compras
34587	Pedro	50
1022305	Maria	48
493942	Juana	125
45307	Lola	50
21566	Carlos	30
12548	Miguel	50
568978	Antonio	70

Vendedores		
Codigo	Nombre	Num_ventas
65154	Luis	65
132155	Juana	15
548965	Maria	23
213564	David	45
568655	Amparo	48
215685	Carlos	56
995485	Teresa	48

```
SELECT *  
FROM compradores C  
FULL JOIN vendedores V  
ON C.nombre = V.nombre;
```



c.Codigo	c.Nombre	Num_compras	v.Codigo	v.Nombre	Num_ventas
34587	Pedro	50	NULL	NULL	NULL
1022305	Maria	48	548965	Maria	23
493942	Juana	125	132155	Juana	15
45307	Lola	50	NULL	NULL	NULL
21566	Carlos	30	215685	Carlos	56
12548	Miguel	50	NULL	NULL	NULL
568978	Antonio	70	NULL	NULL	NULL
NULL	NULL	NULL	65154	Luis	65
NULL	NULL	NULL	213564	David	45
NULL	NULL	NULL	568655	Amparo	48
NULL	NULL	NULL	995485	Teresa	48

FULL JOIN (MySQL)

En algunas versiones de MySQL, no se puede usar FULL JOIN. Pero si niños damos cuenta, un FULL JOIN es la unión entre un LEFT JOIN y un RIGHT JOIN iguales (con la misma condición).

Por eso, como en nuestro ejemplo tanto el número de columnas como el tipo son iguales, podemos expresarlo de la manera:

```
SELECT *  
FROM compradores C  
FULL JOIN vendedores V  
ON C.nombre = V.nombre;
```



```
(SELECT *  
FROM compradores C  
LEFT JOIN vendedores V  
ON C.nombre = V.nombre)  
UNION  
(SELECT *  
FROM compradores C  
RIGHT JOIN vendedores V  
ON C.nombre = V.nombre  
);
```

OUTER JOIN

LIBROS			
Codigo	Titulo	Num_paginas	Editorial
34587	Int. Artificial	50	Paraninfo
1022305	Concep. Y Dis.	48	Rama
493942	Turbo C++	125	Mc Graw-Hill
45307	Virus Informát.	50	NULL
112313	Sist. Informac.	358	Rama

EDITORIALES			
Nombre_e	Direccion	Pais	Ciudad
Universal Books	Brown Sq. 23	EEUU	Los Ángeles
Rama	Canillas, 144	España	Madrid
Mc Graw-Hill	Basauri, 17	España	Madrid
Paraninfo	Virtudes, 7	España	Madrid

Los OUTER JOIN sirven también para recuperar las filas que tienen un campo que está en una tabla, pero no en la otra.

Por ejemplo, si se quieren saber las editoriales que no tienen libros, podemos hacer un RIGHT JOIN, o bien hacer un LEFT JOIN entre editorial y libros, para saber todas las editoriales y los libros que tienen, y también nos mostrará las editoriales que no tienen libros. Si luego eliminamos las editoriales que tienen libros (es decir, las filas en las que el libro exista), tendremos las editoriales que no tienen libros.

```
SELECT E.*
FROM EDITORIALES E
LEFT JOIN LIBROS L
ON E.nombre_e = L.editorial
WHERE L.editorial IS NULL;
```

LEFT
JOIN

Nombre_e	Direccion	Pais	Ciudad	Codigo	Titulo	Num_paginas	Editorial
Universal Books	Brown Sq. 23	EEUU	Los Ángeles	NULL	NULL	NULL	NULL
Rama	Canillas, 144	España	Madrid	112313	Sist. Informac.	358	Rama
Rama	Canillas, 144	España	Madrid	1022305	Concep. Y Dis.	48	Rama
Mc Graw-Hill	Basauri, 17	España	Madrid	493942	Turbo C++	125	Mc Graw-Hill
Paraninfo	Virtudes, 7	España	Madrid	34587	Int. Artificial	50	Paraninfo
Paraninfo	Virtudes, 7	España	Madrid	21566	Optimización	30	Paraninfo
Paraninfo	Virtudes, 7	España	Madrid	12548	Seguridad	50	Paraninfo
Paraninfo	Virtudes, 7	España	Madrid	568978	Hardware	70	Paraninfo

WHERE + proyección

Nombre_e	Direccion	Pais	Ciudad
Universal Books	Brown Sq. 23	EEUU	Los Ángeles

PRODUCTO CARTESIANO Y JOINS

Hemos visto como en la sección FROM de una consulta se pueden usar tanto el producto cartesiano como el JOIN. También se pueden usar ambos simultáneamente, y, en ese caso, primero se realizan los JOINS, y luego se realizan los productos cartesianos. Veamos unos ejemplos:

Consulta	Modo de ejecución
<pre>SELECT * FROM a INNER JOIN b ON a.campo= b.campo, c;</pre>	Primero se realiza el inner join entre a y b, y sobre el resultado, se hace el producto cartesiano con c.
<pre>SELECT * FROM a, b INNER JOIN c ON b.campo= c.campo, d;</pre>	Primero se realiza el inner join de b y c. Luego, se hace el producto cartesiano de a y el resultado de b y c, y por último, sobre ese resultado se hace el producto cartesiano con d.

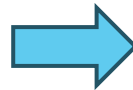
TABLAS DERIVADAS

Como ya vimos cuando explicamos las consultas monotablas, en la sección FROM se pueden usar resultados de consultas como tablas. Lo mismo aplica en las consultas multitabla. Esta subconsulta puede contener todas las secciones de una consulta excepto el ORDER BY y el LIMIT.

Por ejemplo, consideremos la siguiente tabla, similar a la tabla DetallePedidos del modelo de jardinería con el que trabajamos en clase. Imaginad que queremos calcular el ticket (el pedido en el caso de jardinería) de mayor importe. Ya hemos visto que las funciones resumen no se pueden anidar, por lo que en muchas ocasiones recurriremos a las subconsultas o a las tablas derivadas para lograr obtener nuestros resultados:

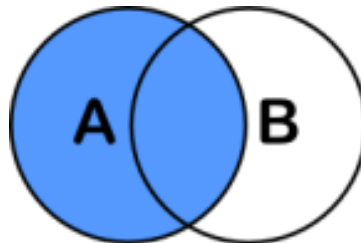


SELECT MAX(SUM (cantidad * PrecioUnitario))
FROM lineaTicket
GROUP BY codigoTicket;

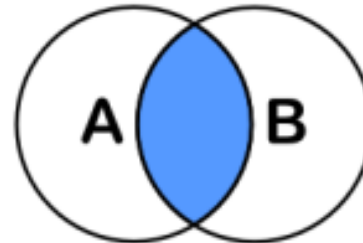


SELECT MAX(totalTicket.total)
FROM (SELECT SUM (cantidad * PrecioUnitario) as total,
codigoTicket
FROM lineaTicket
GROUP BY codigoTicket) AS totalTicket;

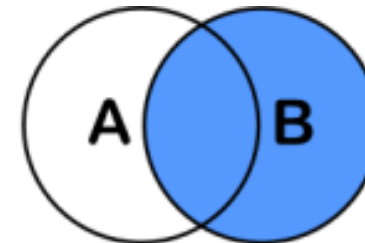
RESUMEN TIPOS DE JOIN



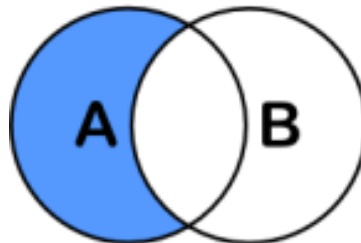
```
SELECT <auswahl>  
FROM tabelleA A  
LEFT JOIN tabelleB B  
ON A.key = B.key
```



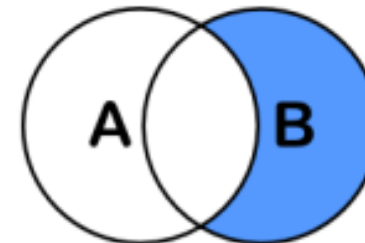
```
SELECT <auswahl>  
FROM tabelleA A  
INNER JOIN tabelleB B  
ON A.key = B.key
```



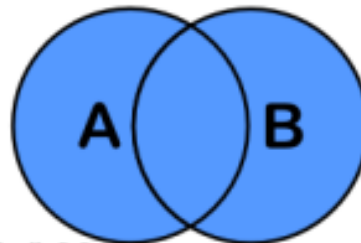
```
SELECT <auswahl>  
FROM tabelleA A  
RIGHT JOIN tabelleB B  
ON A.key = B.key
```



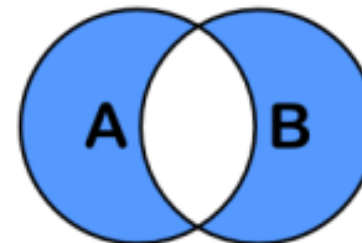
```
SELECT <auswahl>  
FROM tabelleA A  
LEFT JOIN tabelleB B  
ON A.key = B.key  
WHERE B.key IS NULL
```



```
SELECT <auswahl>  
FROM tabelleA A  
RIGHT JOIN tabelleB B  
ON A.key = B.key  
WHERE A.key IS NULL
```



```
SELECT <auswahl>  
FROM tabelleA A  
FULL OUTER JOIN tabelleB B  
ON A.key = B.key
```



```
SELECT <auswahl>  
FROM tabelleA A  
FULL OUTER JOIN tabelleB B  
ON A.key = B.key  
WHERE A.key IS NULL  
OR B.key IS NULL
```

RESUMEN SENTENCIA SELECT

SELECT [**DISTINCT**] *select_expr* [, *select_expr* ...]

FROM *table_references*

[**WHERE** *filtro*]

[**GROUP BY** *expr* [, *expr* ...]

[**HAVING** *filtro_grupos*]]

[**ORDER BY** { *col_name* | *expr* | *position* } [**ASC** | **DESC**] , ...]

[**LIMIT** [*desplazamiento* ,] *nfilas*]

Se pueden unir varios bloques de este tipo con:
UNION
INTERSECT
EXCEPT

EJECUCIÓN DE UNA SENTENCIA SELECT

1. Si existen subconsultas que no referencien a columnas de la consulta principal, se ejecutan las subconsultas primero siguiendo los pasos aquí descritos.
2. Se toma la tabla indicada en el FROM. Se aplican los JOINS si existen, y los productos cartesianos después (si existen). Si alguna de las tablas es una tabla derivada, se calcula previamente según los pasos aquí descritos.
3. Para cada fila, se aplican las condiciones indicadas en el WHERE. Si en esa fila no se cumple (resultado falso) las condiciones, se elimina la fila del conjunto de resultados. Si hay subconsultas en la parte WHERE que referencian a columnas de la consulta principal, se ejecutan en este momento.
4. Se aplica el GROUP BY, agrupando las filas por las columnas indicadas.
5. Se aplica el HAVING, eliminando las filas agrupadas que no cumplan la condición. Si hay subconsultas en la parte HAVING que referencian a columnas de la consulta principal, se ejecutan en este momento.
6. Para cada fila, se calculan las columnas que se devolverán, indicadas en la parte SELECT. Si hay subconsultas en la parte SELECT que referencian a columnas de la consulta principal, se ejecutan en este momento.
7. Se aplican la UNION, INTERSECCIÓN, DIFERENCIA...
8. Se ordenan las filas según se indica en la parte ORDER BY
9. Solo se muestran las filas que cumplan la condición indicada en LIMIT

Preguntas

